Republic of Turkey

Kastamonu University

Faculty of Engineering and Architecture

Department of Computer Engineering

OBJECT-ORIENTED PROGRAMMING

PROJECT REPORT

Project Owner	Gizem KARAASLAN
School Number	214410036
Advisor	Ahmet Nusret ÖZALP
Course Code	BSM204
Date	30.05.2026

API Call Monitoring System

API Call Monitoring and Threat Detection System: A Polymorphic, Rule-Based Behavioral Analysis Engine in C++17

Abstract—This paper presents the design and implementation of a behavioral API call monitoring and threat detection system developed in C++17 using the four core principles of object-oriented programming: abstraction, inheritance, polymorphism, and encapsulation. The system intercepts simulated Windows API call sequences, evaluates them against a set of pluggable detection rules, and emits severity-labeled alerts in real time. Three concrete rule classes—RansomwareRule, CodeInjectionRule, and SuspiciousNetworkRule—are derived from a pure virtual base class DetectionRule and are orchestrated by a DetectionEngine that applies each rule polymorphically to an accumulating call history. Four simulation scenarios (ransomware encryption, DLL injection, data exfiltration, and a benign control case) validate the detection logic. The project compiles under C++17 with no external dependencies and serves as a comprehensive pedagogical demonstration of OOP patterns in a security-relevant systems programming context.

Keywords—behavioral detection, API monitoring, object-oriented programming, polymorphism, inheritance, abstraction, encapsulation, ransomware detection, code injection, data exfiltration, C++17, threat detection

I. INTRODUCTION

The proliferation of sophisticated malware has made static signature-based antivirus detection increasingly insufficient. Modern threats—including ransomware, process injection toolkits, and command-and-control (C2) implants—are routinely obfuscated, packed, or polymorphically mutated to evade byte-pattern matching. Behavioral detection, which focuses on what a program does rather than what it looks like, has consequently become the cornerstone of contemporary endpoint protection platforms [1].

At the operating system level, the observable unit of malicious behavior is the Windows API call. Ransomware is characterized by bursts of WriteFile calls to paths with encryption-suffix extensions and by Shadow Copy deletion via vssadmin or wmic. DLL injection follows a canonical three-API sequence: OpenProcess, VirtualAllocEx, and WriteProcessMemory or CreateRemoteThread. Data exfiltration combines ReadFile operations on sensitive paths with outbound network activity on well-known C2 ports such as 4444 or 1337 [2].

This paper describes a self-contained C++17 system that models these behavioral patterns as pluggable rule objects derived from a common abstract base class. The engine maintains a per-process API call history and evaluates each new call against all registered rules, producing severity-labeled alerts. Four OOP principles are explicitly exercised: abstraction through the pure virtual DetectionRule interface, inheritance through three concrete rule subclasses, polymorphism through virtual dispatch in DetectionEngine::processCall(), and encapsulation through private member access control in both DetectionEngine and DetectionRule.

The remainder of this paper is organized as follows. Section II reviews related work on behavioral detection and OOP in security tools. Section III describes the data structures. Section IV details the class hierarchy. Section V describes the detection engine. Section VI presents the simulation scenarios and results. Section VII discusses the design and limitations. Section VIII concludes.

II. RELATED WORK

A. Behavioral Malware Detection

Behavioral analysis of malware has been studied extensively since the introduction of dynamic analysis sandboxes in the early 2000s [3]. Systems such as Cuckoo Sandbox [4] and TTAnalyze [5] execute malware samples in isolated virtual machines and record API call sequences, network traffic, and file system modifications. The resulting behavioral reports are then classified using rule sets, decision trees, or neural networks.

API call n-gram models have demonstrated strong classification accuracy for malware family identification [6]. More recent work has applied recurrent neural networks and transformer architectures to API call sequences, treating them analogously to natural language tokens [7]. The present tool uses an interpretable rule-based approach, which, while less general than learned models, is fully auditable and requires no training data.

B. Object-Oriented Design in Security Software

Security tools have historically been written in C for performance and low-level access reasons. The adoption of C++ in security tooling has been driven by the need to manage increasing code complexity while maintaining performance. Notable examples include the YARA rule engine, which uses C structures with function pointers that approximate OOP dispatch, and the Volatility memory forensics framework (Python), which uses an explicit class hierarchy for OS artifact modeling [8].

The Strategy and Template Method design patterns, both of which are realized in this work through the DetectionRule hierarchy, are widely recommended for extensible security policy engines [9]. The Strategy pattern allows detection logic to be selected and swapped at runtime without modifying the engine, while the Template Method pattern defines the invariant check-then-report skeleton in the base class.

C. API Call Monitoring

User-mode API call interception is implemented in production security products through inline hooks (as described in companion work [10]), import address table (IAT) patching, and kernel callback registration [11]. For educational purposes, this tool simulates the monitoring layer by accepting pre-constructed APICall objects, allowing the behavioral logic to be studied and extended without requiring kernel-mode access or platform-specific hooking infrastructure.

III. DATA STRUCTURES

A. APICall Structure

The APICall struct encapsulates the data associated with a single observed API invocation. It stores the name of the originating process (processName), the name of the called Windows API function (apiName), a vector of string arguments (args), and an ISO 8601 timestamp (timestamp). The constructor accepts an optional timestamp parameter; if omitted, the current system time is formatted and assigned automatically using std::strftime. A toString() helper method serializes the struct to a human-readable log line for diagnostic output.

The argument vector is intentionally untyped (std::vector<std::string>) to accommodate the heterogeneous parameter lists of different Windows APIs without requiring per-API specialization. Rule classes access arguments through the protected argContains() helper in DetectionRule, which performs substring matching across all elements of the args vector.

B. Alert Structure and Severity Enum

Detected threats are represented by the Alert struct, which records the name of the triggering rule (ruleName), a four-level severity classification (Severity::LOW, MEDIUM, HIGH, CRITICAL), a human-readable description, and a non-owning raw pointer to the APICall that triggered the alert. The non-owning pointer is safe because Alert objects are consumed within the same stack frame as the APICall they reference and are never stored beyond the lifetime of their trigger.

The Severity enum class is mapped to ANSI color codes for console output (green for LOW, yellow for MEDIUM, red for HIGH, magenta for CRITICAL) through the severityColor() free function, providing immediate visual triage for the operator.

IV. CLASS HIERARCHY

A. DetectionRule — Abstract Base Class

DetectionRule defines the interface that all detection rules must implement. Its sole pure virtual method, checkBehavior(), accepts the full call history accumulated so far and the current APICall under evaluation, and returns a (possibly empty) vector of Alert objects. The virtual destructor ensures correct destruction of derived objects through base-class pointers. Two protected helper methods—countAPI() and argContains()—provide reusable query primitives that avoid code duplication across subclasses.

The member variables ruleName_ and description_ are declared protected rather than private to permit read access in subclass constructors without exposing public setters. The public getName() and getDescription() getters provide read-only external access, completing the encapsulation boundary.

TABLE I. Class Hierarchy and OOP Principles

Class	Type	OOP Principle(s) Demonstrated
`DetectionRule`	Abstract base class	Abstraction (pure virtual interface), Encapsulation (protected members)
RansomwareRule	Concrete derived class	Inheritance, Polymorphism (checkBehavior override)
CodeInjectionRule	Concrete derived class	Inheritance, Polymorphism (checkBehavior override)
SuspiciousNetworkRule	Concrete derived class	Inheritance, Polymorphism (checkBehavior override)
`DetectionEngine`	Orchestration class	Encapsulation (private rules_ / history_), Polymorphism (virtual dispatch)
`APICall`	Plain data structure	Encapsulation (data aggregation)
Alert	Plain data structure	Encapsulation (result aggregation)

B. RansomwareRule

RansomwareRule implements three detection sub-rules. The first sub-rule fires when a CreateProcess or ShellExecute call contains the strings "vssadmin", "wmic", or "shadowcopy" in its argument list, indicating an attempt to delete Volume Shadow Copies—a near-universal ransomware behavior—and assigns CRITICAL severity. The second sub-rule detects MoveFile or CopyFile calls with encryption-suffix destinations (.encrypted, .locked, .cry, .ransom) and assigns HIGH severity. The third sub-rule counts WriteFile calls for the same process in the history; once the configurable writeThreshold is exceeded, a HIGH-severity bulk-encryption alert is raised. The threshold is set to four in the demonstration but is fully parameterizable through the constructor.

C. CodeInjectionRule

CodeInjectionRule detects the canonical three-step DLL/shellcode injection sequence. Upon observing VirtualAllocEx from a process that has previously called OpenProcess, a HIGH-severity alert is raised indicating remote memory allocation. Upon observing WriteProcessMemory from a process with both OpenProcess and VirtualAllocEx in its history, a CRITICAL alert is raised confirming the complete injection triad. CreateRemoteThread always raises a CRITICAL alert independently, as it represents the execution stage of injection regardless of preceding API calls.

D. SuspiciousNetworkRule

SuspiciousNetworkRule addresses data exfiltration scenarios. It monitors connect and WSASend calls and raises a MEDIUM-severity alert when a prior ReadFile has been observed from the same process, indicating that data read from disk may be in transit. Additionally, connections to ports 4444, 1337, and 31337—well-known Metasploit and backdoor ports—independently trigger a HIGH-severity C2 communication alert regardless of prior file reads. The two sub-rules may co-fire on the same call, producing multiple alerts with different descriptions and severities.

V. DETECTION ENGINE

DetectionEngine is the central orchestrating class. It maintains two private members: rules_, a vector of unique_ptr<DetectionRule> that holds ownership of all registered rule objects, and history_, a vector of APICall representing the stream of events processed so far. The addRule() method accepts a unique_ptr by value and moves it into rules_, transferring ownership to the engine and preventing double-free errors.

The processCall() method implements the core dispatch loop. For each registered rule, it invokes rule->checkBehavior(history_, call) through a base-class pointer. Because checkBehavior() is declared virtual in DetectionRule, the C++ runtime dispatches to the correct concrete override without the engine needing any knowledge of the concrete type. This is the polymorphism principle in action: the engine is closed for modification but open for extension—a direct realization of the Open/Closed Principle [12]. After all rules have been evaluated, the current call is appended to history_ for use by subsequent calls.

The reset() method clears history_ without affecting rules_, allowing the same engine instance to process multiple independent scenarios in sequence, as demonstrated in main(). The getTotalCallsProcessed() and getRuleCount() accessors provide read-only statistics without exposing the internal containers.

VI. SIMULATION SCENARIOS AND RESULTS

A. Scenario 1: Ransomware (cryptolocker.exe)

Eight API calls simulate a ransomware execution chain. The first call opens a document, the second reads it, and the next four WriteFile calls write encrypted versions with the .encrypted suffix. By the fourth WriteFile, the bulk-encryption threshold (set to four) is exceeded and a HIGH alert fires. A MoveFile call to an .encrypted destination then triggers a HIGH file-renaming alert. Finally, a CreateProcess call containing "vssadmin delete shadows" raises a CRITICAL Shadow Copy deletion alert. The scenario demonstrates that multiple sub-rules within RansomwareRule can fire independently on different calls within the same session.

B. Scenario 2: Process Injection (injector.exe → svchost.exe)

Four API calls replicate the classic OpenProcess → VirtualAllocEx → WriteProcessMemory → CreateRemoteThread injection sequence. After OpenProcess, no alert fires. VirtualAllocEx triggers a HIGH alert because OpenProcess is already in history. WriteProcessMemory triggers a CRITICAL alert as the third member of the injection triad. CreateRemoteThread triggers a second CRITICAL alert for remote thread creation. The scenario validates that stateful, history-aware detection correctly fires only when the full prerequisite chain is established.

C. Scenario 3: Data Exfiltration (spyware.exe)

spyware.exe reads two sensitive files (passwords.txt and creditcards.csv) and then initiates a TCP connection to 192.168.100.55:4444. The connect call triggers two simultaneous alerts: a MEDIUM exfiltration alert (because ReadFile precedes it) and a HIGH C2 port alert (because port 4444 is on the known-bad list). The subsequent WSASend call repeats both alerts as it also satisfies both sub-rules. The scenario demonstrates the co-firing of independent sub-rules within the same rule class.

D. Scenario 4: Clean Application (notepad.exe)

notepad.exe performs four operations: OpenFile, ReadFile, WriteFile, and CloseFile on a single text file. None of these calls satisfies any rule condition: the single WriteFile does not reach the ransomware threshold, no process injection APIs are present, and no network calls follow the file read. No alerts are generated, confirming that the detection logic does not produce false positives for benign single-file editing behavior.

TABLE II. Scenario Summary and Alert Outcomes

Scenario	Process	API Calls	Alerts Generated
1 — Ransomware	cryptolocker.exe	8	HIGH (bulk write ×3, rename), CRITICAL (shadow copy delete)
2 — Injection	injector.exe	4	HIGH (VirtualAllocEx), CRITICAL (WriteProcessMemory, CreateRemoteThread)
3 — Exfiltration	spyware.exe	4	MEDIUM (exfiltration ×2), HIGH (C2 port ×2)
4 — Clean	notepad.exe	4	None (true negative confirmed)

VII. DISCUSSION

A. OOP Design Evaluation

The four OOP principles are realized concretely and non-trivially in this project. Abstraction is demonstrated by DetectionRule, which defines what a detection rule must do (evaluate behavior and return alerts) without prescribing how. Inheritance is demonstrated by the three derived rule classes, which reuse the countAPI() and argContains() helpers from the base class while providing entirely distinct checkBehavior() implementations. Polymorphism is demonstrated by DetectionEngine::processCall(), which invokes the correct rule implementation through a base-class pointer with zero engine-side conditionals. Encapsulation is demonstrated by the private rules_ and history_ members of DetectionEngine, which are accessible only through the addRule(), processCall(), and reset() interface methods.

Memory management follows modern C++17 idioms throughout. unique_ptr<DetectionRule> in the rules_ vector guarantees that each rule object is destroyed exactly once when the engine goes out of scope, with no manual delete calls required. The non-owning raw pointer in Alert is an intentional design

choice: alert objects are value-copied into the returned vector and consumed immediately by the caller, so shared ownership semantics would add complexity without benefit.

B. Extensibility

Adding a new detection rule requires only: (1) deriving a new class from DetectionRule; (2) implementing checkBehavior(); and (3) registering an instance with engine.addRule(). No modification to DetectionEngine or to any existing rule class is necessary. This extensibility is a direct consequence of the Open/Closed Principle realized through the virtual dispatch mechanism. Production deployment would typically load rule classes from shared libraries at runtime, allowing security researchers to ship rule updates without recompiling the engine binary.

C. Limitations

Several limitations bound the scope of the current implementation. First, the API call stream is synthetic; a production system would need to integrate with a kernel-mode monitoring driver or a user-mode hooking layer to intercept live API calls. Second, the detection rules use simple count thresholds and string matching; adversaries can trivially evade them by spreading WriteFile calls across longer time windows or by encoding argument strings. Third, the history_ vector is unbounded and grows linearly with the number of processed calls; a sliding time window with configurable retention would be necessary for long-running deployment.

Fourth, the tool processes all processes in a single shared history per DetectionEngine instance. In the simulation, engine.reset() is called between scenarios to avoid cross-contamination; a production system would maintain per-process history maps. Fifth, the console color codes use ANSI escape sequences, which are not supported natively on Windows terminals prior to Windows 10 Anniversary Update without enabling virtual terminal processing via SetConsoleMode.

D. Comparison with Production Systems

The architecture of this tool shares structural similarities with the rule engine in Snort and Suricata network intrusion detection systems, which also maintain stateful session histories and evaluate incoming packets against a set of pluggable detection rules [13]. The primary differences are scale (millions of packets per second versus tens of API calls per scenario), rule language (custom domain-specific languages versus C++ virtual methods), and deployment context (network versus host). The pedagogical value of this tool lies in making the conceptual machinery of such systems directly readable and modifiable without domain-specific language overhead.

VIII. CONCLUSION

This paper presented a behavioral API call monitoring and threat detection system implemented in C++17 that explicitly demonstrates all four pillars of object-oriented programming. The abstract DetectionRule base class provides a stable interface for an open-ended set of behavioral rules; three concrete subclasses—covering ransomware, code injection, and network exfiltration—implement distinct detection logics while reusing shared utility methods; DetectionEngine applies all rules polymorphically through virtual dispatch; and private member access throughout the codebase enforces clean encapsulation

boundaries. Four simulation scenarios validate correct detection of known malicious API patterns and confirm the absence of false positives on benign workloads. The system is fully self-contained, compilable under C++17 with no external dependencies, and serves as a pedagogically complete reference for OOP-based security tool design.

ACKNOWLEDGMENT

The authors thank the reviewers for their constructive feedback and the security research community for publicly documenting the behavioral patterns of malware families referenced in the simulation scenarios. This work was conducted as part of an Object-Oriented Programming course project and received no external funding.

REFERENCES

[1] M. Sikorski and A. Honig, Practical Malware Analysis. San Francisco, CA: No Starch Press, 2012.

[2] FireEye Mandiant, "M-Trends 2023: Special Report," FireEye, Inc., 2023. [Online]. Available: https://www.mandiant.com/m-trends

[3] C. Willems, T. Holz, and F. Freiling, "Toward automated dynamic malware analysis using CWSandbox," IEEE Secur. Priv., vol. 5, no. 2, pp. 32–39, Mar./Apr. 2007.

[4] C. Guarnieri et al., "Cuckoo sandbox: Automated malware analysis," in Proc. 6th USENIX Workshop Large-Scale Exploits Emergent Threats, Washington, DC, 2013.

[5] U. Bayer et al., "TTAnalyze: A tool for analyzing malware," in Proc. 15th Eur. Inst. Comput. Antivirus Res. Conf., Hamburg, Germany, 2006.

[6] K. Rieck et al., "Automatic analysis of malware behavior using machine learning," J. Comput. Secur., vol. 19, no. 4, pp. 639–668, 2011.

[7] Z. Fang et al., "MalBERT: Malware detection using bidirectional encoder representations from transformers," Comput. Secur., vol. 128, p. 103176, 2023.

[8] A. Walters and N. Petroni, "Volatools: Integrating volatile memory forensics into the digital investigation process," in Proc. BlackHat Federal, Washington, DC, 2007.

[9] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1994.

[10] [Author(s)], "Inline hook detection tool: An OOP-based C++ demonstration," in Proc. IEEE Conference, City, Country, 20XX.

[11] J. Richter and C. Nasarre, Windows via C/C++, 5th ed. Redmond, WA: Microsoft Press, 2008.

[12] R. C. Martin, Clean Architecture: A Craftsman’s Guide to Software Structure and Design. Upper Saddle River, NJ: Prentice Hall, 2017.

[13] M. Roesch, "Snort: Lightweight intrusion detection for networks," in Proc. 13th USENIX Syst. Admin. Conf., Seattle, WA, 1999, pp. 229–238.